

AI TRAVEL AGENT

College Project Report

Artificial Intelligence and Natural Language Processing

Conversational travel planning with constraint optimisation and route ordering

October 2026

TABLE OF CONTENTS

Chapter 1	Introduction	3
Chapter 2	Objectives	4
Chapter 3	System Description and NLP Approach.....	5
Chapter 4	AI and NLP Pipeline and Architecture.....	7
Chapter 5	Mathematical Formulation and Algorithms	8
Chapter 6	Tech Stack and Packages.....	12
Chapter 7	Database and Knowledge Base Structure	14
Chapter 8	Modules and Implementation	15
Chapter 9	Output Screenshots	20
Chapter 10	Conclusion and Future Enhancement	45
	References and source materials	46

CHAPTER 1

INTRODUCTION

1.1 Background

Planning a trip requires coordinating destinations, travel dates, transport, accommodation, food, sightseeing and expenses. These decisions depend on one another. A destination that appears attractive may be too far from the origin, an outdoor activity may be unsuitable during rain, and a day containing several long treks may be physically unrealistic. Searching separate websites does not automatically produce a consistent day-wise plan.

1.2 Problem Statement

Travellers usually express requirements informally, such as “Plan a relaxed three-day trip from Chennai to Ooty for two people.” A useful planning system must identify these details, ask for missing information, retrieve relevant places and turn preferences into a feasible schedule. It must also distinguish verified attraction facts from estimated duration, travel distance, cost and weather suitability.

1.3 Project Overview

The AI Travel Agent provides a conversational interface for personalised trip planning. Its React application communicates with a FastAPI backend and a Python planning agent. The agent combines natural language understanding, destination discovery, external location and weather services, activity classification, constraint-based scheduling and route optimisation. Trip state and conversation history are stored in SQLite so users can reopen saved sessions.

1.4 Scope

The project focuses on itinerary assistance, with curated Indian destinations and attraction data supplemented by OpenStreetMap. It supports destination selection, recommendations, day-wise plans, map views and budget estimates. OR-Tools assigns attractions to days, while route-ordering algorithms reduce travel within each day. Prices are planning assumptions rather than live booking quotations. Weather can be a short-range forecast or a historical climate estimate, depending on the trip dates. The resulting itinerary is a decision aid that travellers can review and edit before travel.

CHAPTER 2

OBJECTIVES

2.1 Primary Objective

The primary objective is to convert a traveller's natural-language requirements into a personalised and explainable itinerary by combining NLP, travel data retrieval and optimisation.

2.2 Specific Objectives

1. Understand travel requests. Extract origin, destination, dates, budget, traveller count, transport mode and preferences from conversational messages.
2. Identify user intent. Recognise requests for recommendations, itinerary generation, weather, transport, budget information and changes to the plan.
3. Discover relevant destinations. Use category and proximity search to suggest destinations when the user has not selected a specific place.
4. Build feasible schedules. Assign attractions to days while respecting stop limits, daily effort, intensity, trek limits and supported opening-hour information.
5. Improve route order. Use road-distance data when available and apply A* or nearest-neighbour with 2-opt to sequence daily visits.
6. Personalise recommendations. Consider interests, pace, accessibility requirements, weather suitability and crowd-avoidance preferences.
7. Present useful outputs. Display recommendations, daily timelines, stay options, budget estimates and geographic locations in the web interface.
8. Preserve sessions and provenance. Save trip state and messages, and label attraction facts and estimated values with their sources.

CHAPTER 3

SYSTEM DESCRIPTION AND NLP APPROACH

3.1 System Description

The application has a browser interface, an API layer and a planning layer. React manages the conversation and trip views. FastAPI exposes session creation, chat, recommendations, itinerary movement and map endpoints. The Python agent owns the conversational state and invokes the tools needed to complete a travel request. Pydantic models provide a consistent representation of trip dates, destinations, preferences and generated outputs.

3.2 Named Entity Recognition and Slot Extraction

The NER module extracts structured trip slots before the agent decides its next action. It uses spaCy’s English model when available, with regular-expression rules that continue to work when the model is missing. The rules cover Indian date formats, amounts such as “40k INR”, durations, traveller counts, solo travel, origins and transport keywords. Place names are resolved through geocoding and destination-specific logic.

For example, “From Chennai to Ooty for two people, budget 30000 INR, from 10/10/2026 to 12/10/2026” supplies an origin, destination, group size, budget and date interval. The extracted slots are merged into the session state. Missing or uncertain details are handled through further conversation rather than silently inventing values.

3.3 Intent Classification

Intent recognition has two layers. High-precision rules identify clear messages such as greetings, numbered destination choices and explicit scheduling commands. Other messages pass through a local classifier trained on labelled examples. Word unigrams and bigrams are combined with character n-grams using TF-IDF, and Logistic Regression predicts the intent and its confidence.

Intent categories include providing trip details, requesting recommendations, building an itinerary, changing pace, removing a stop, asking about budget, confirming a destination, asking about transport, asking about weather, greeting and other requests. Rule matches are trusted directly; classifier predictions require sufficient confidence before they trigger a shortcut.

3.4 Semantic Routing and Tool Dispatch

A hybrid router determines whether a message should use the deterministic NLP tool engine or the LLM conversation engine. It compares local FastEmbed sentence embeddings against example requests using cosine similarity and also considers intent confidence, message complexity and the current trip state. A TF-IDF fallback is available when embedding initialisation fails.

The deterministic tool router checks whether the required state is available before calling a tool. A weather request, for example, needs a resolved destination and relevant dates. Simple commands can therefore be processed without an LLM tool-calling round. Open-ended comparisons, ambiguous requests and conversational repair are passed to the orchestrator's LLM path.

3.5 LLM Orchestration

The orchestrator maintains conversation history, fits messages within a token budget and coordinates tool results with the selected language model. The shared client supports Groq, NVIDIA through an OpenAI-compatible API, and a local OpenAI-compatible service such as Ollama. Model selection and credentials are environment settings; the application does not require one fixed model for every deployment.

Activity classification is another LLM-assisted task. It estimates visit duration, activity class, intensity and related attributes when stronger place-specific information is unavailable. Returned values are validated and bounded before scheduling. Cache reuse reduces repeated requests, while conservative defaults allow the planner to continue when classification fails.

3.6 Data Grounding and Limitations

Destination and attraction information comes from curated records and external services. The model does not serve as the sole source of attraction facts. Approved profiles, OpenStreetMap tags, category rules and model estimates are distinguished through provenance fields. Missing prices and accessibility remain unknown unless a source supplies them. NLP extraction can still misinterpret uncommon place names, indirect phrasing or unsupported language, so the interface and conversation should allow the traveller to inspect and correct the plan.

CHAPTER 4

AI AND NLP PIPELINE AND ARCHITECTURE

CHAPTER 5

MATHEMATICAL FORMULATION AND ALGORITHMS

5.1 Text Representation and Intent Prediction

The classifier represents each user message as a vector of word and character features. For a term t in message q , the following expressions summarise the smoothed TF-IDF weighting used by the vectoriser. N is the number of training messages and $df(t)$ is the number containing the term. The resulting feature vector is normalised before classification.

$$idf_t = \ln\left(\frac{1 + N}{1 + df(t)}\right) + 1$$

$$v_{t,q} = tf_{t,q} \times idf_t$$

$$P(y = k | v) = \frac{\exp(w_k^T v + b_k)}{\sum_j \exp(w_j^T v + b_j)}$$

The predicted intent is the class with the highest probability. The NLU entry point treats rule matches as high confidence and requires a classifier confidence of at least 0.70 for a high-confidence result. This threshold controls dispatch; it is not a measured accuracy score.

5.2 Semantic Similarity

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \|v\|}$$

The semantic router compares a message embedding u with route exemplar embeddings v . Larger cosine similarity indicates stronger alignment with an exemplar. Similarity is combined with confidence and dialogue-state checks to choose the NLP or LLM path. The router uses pretrained local embeddings rather than training a language model from the trip database.

5.3 Geographic Distance

$$a = \sin^2(\Delta\phi / 2) + \cos(\phi_1) \cos(\phi_2) \sin^2(\Delta\lambda / 2)$$

$$d = 2R \arcsin(\sqrt{a})$$

Here ϕ denotes latitude, λ denotes longitude, angles are in radians, and R is approximately 6371 km. Haversine distance supports nearby destination search and radius filtering. It measures straight-line surface distance; road-distance services are preferred for travel planning. The scheduler uses a labelled approximation of 1.4 times Haversine distance when its road matrix is unavailable.

5.4 Attraction Assignment Variables

Let I be the set of candidate attractions and D the set of trip days. A binary variable $x_{i,d}$ equals 1 when attraction i is assigned to day d and 0 otherwise. Let e_i be visit effort in minutes, q_i be intensity and t_i indicate a trek. Each attraction can be omitted if the complete pool cannot satisfy the traveller's limits.

$$x_{i,d} \in \{0, 1\}$$

$$\sum_{d \in D} x_{i,d} \leq 1 \text{ for every attraction } i$$

$$\sum_{i \in I} x_{i,d} \leq M \text{ for every day } d$$

The first inequality prevents duplicate visits. The second limits the daily number of stops to M . Daily visit effort and intensity are bounded separately so a short sightseeing stop and a long trek are not treated as equivalent.

$$\sum_{i \in I} e_i x_{i,d} \leq E_{max}$$

$$\sum_{i \in I} q_i x_{i,d} \leq Q_{max}$$

The implemented default effort caps are 240, 330 and 420 minutes for relaxed, balanced and packed pace. The documented intensity defaults are 5, 8 and 11 respectively. These limits constrain the planning model; they are not medical assessments of an individual traveller.

5.5 Trek and Availability Constraints

$$\sum_{i \in I} t_i x_{i,d} \leq 1$$

$$\sum_{d \in D} \sum_{i \in I} t_i x_{i,d} \leq T_{max}$$

At most one trek is allowed per day, and the total number of treks is limited by the selected adventure level. A supported opening-hours expression that rules out a visit on a particular date forces $x_{i,d}$ to zero. Extremely distant attraction pairs cannot share a day. Complex opening-hours expressions remain unverified rather than being interpreted as guaranteed availability.

5.6 CP SAT Objective Function

OR-Tools CP-SAT chooses a feasible assignment using a weighted objective. The following notation summarises the implemented components. S is the total number of scheduled attractions, R is rain exposure, C is the condition penalty, P is pairwise geographic dispersion, H is nearby-place cohesion, L is stop-count imbalance, E is effort imbalance, V is activity diversity and A penalises excessive repetition of one activity class.

$$\begin{aligned} \min J = & -10000S + w_r R + C + P - w_h H \\ & + w_l L + w_e E - w_v V + w_a A \end{aligned}$$

The large negative coefficient rewards scheduling attractions that satisfy the hard constraints. Rain and condition penalties discourage unsuitable day assignments. Cohesion favours grouping nearby places; dispersion discourages excessive separation. Load and effort terms distribute the workload across days. Diversity rewards a mixture of activity classes instead of repeatedly scheduling the same type of stop.

$$\begin{aligned} S &= \sum_{i \in I} \sum_{d \in D} x_{i,d} \\ L &= \sum_{d \in D} |\sum_{i \in I} x_{i,d} - target| \end{aligned}$$

Pairwise co-assignment uses an auxiliary Boolean variable $b_{i,j}^d$. It is true only when attractions i and j are both scheduled on day d . The equivalent linear inequalities below let CP-SAT express pairwise rewards and penalties.

$$\begin{aligned} b_{i,j}^d &\leq x_{i,d} \quad \text{and} \quad b_{i,j}^d \leq x_{j,d} \\ b_{i,j}^d &\geq x_{i,d} + x_{j,d} - 1 \end{aligned}$$

5.7 Solver and Timeline Construction

The solver runs within a time limit and accepts an optimal or feasible solution. A greedy assignment is used if CP-SAT cannot provide either. After assignment, geographic day ordering and within-day route sequencing produce arrival and departure times. Meals and travel durations are inserted into the timeline, and the result is checked for daily overruns. Current assignment operates on individual attractions with pairwise cohesion; geographic DBSCAN pre-clustering has been removed from this stage.

Budget estimation is a separate calculation. The current CP-SAT assignment should not be described as a proof that the total trip cost is below the user's budget. The displayed estimate helps the traveller compare the plan with available funds.

5.8 A Star Route Ordering

For a day with at most eight stops, A* searches visit sequences that begin and end at the hotel node. A search state contains a visited-stop bitmask and the current stop. The priority combines the distance already travelled with a lower-bound estimate of the remaining route.

$$f(n) = g(n) + h(n)$$

$$h = \min \text{ distance to } U + MST(U) + \min \text{ return distance}$$

U is the unvisited set. The heuristic combines a connection to U, a minimum spanning tree over U, and a connection back to the hotel. For the tree, the smaller of the two directed distances is used. Road-matrix distances determine route quality; an estimated matrix means the resulting order is optimal for that model rather than a guarantee of the shortest real-world road journey.

5.9 Nearest Neighbour and Two Opt

For larger daily groups, nearest-neighbour selects the next unvisited stop with the lowest travel cost. A 2-opt improvement step reverses route segments when doing so reduces the route cost. These heuristics reduce search effort but do not guarantee a globally optimal route. Manual itinerary movement preserves the user's chosen order and retimes the affected day.

5.10 Budget Estimation

$$C_{total} = C_{stay} + C_{food} + C_{local} + C_{entry} + C_{intercity}$$

$$C_{stay} = \max(D - 1, 0) \times 3500 \times \lceil P / 2 \rceil$$

$$C_{food} = D \times P \times 2 \times 400$$

$$C_{local} = D \times 1500$$

D denotes trip days and P denotes travellers. The implemented assumptions are INR 3500 per room-night, INR 400 per meal and INR 1500 per day for local transport. Known entry fees are combined with an INR 100 per-person allowance for each unknown-fee stop. Intercity transport adds a model-based fare or fuel estimate when the selected mode and distance are available. These are explicit assumptions, not live market prices.

CHAPTER 6

TECH STACK AND PACKAGES

6.1 Application and NLP Technologies

Technology or Package	Role in the project
React and React DOM	Browser interface, conversation state and trip views.
Vite	Frontend development server and build tooling.
React Router	Navigation between landing and trip-builder pages.
Leaflet and React Leaflet	Interactive map components and geographic markers.
MapLibre GL and React Map GL	Vector map rendering and map integration.
Python	Planning logic, NLP modules and service clients.
FastAPI and Uvicorn	HTTP API endpoints and ASGI application server.
Pydantic	Trip-state schemas and request/response validation.
spaCy	Optional English named-entity recognition.
scikit-learn	TF-IDF features, Logistic Regression and router fallback.
FastEmbed	Local ONNX sentence embeddings for semantic routing.
Joblib	Persistence of the intent classifier.
Groq SDK	Configured Groq language-model access.
OpenAI SDK	OpenAI-compatible NVIDIA and local model access.

The provider selected at runtime depends on environment configuration. The presence of a package in requirements does not establish that it is used by the active React and FastAPI application.

6.2 Optimisation Data and Supporting Packages

Technology or Service	Purpose or usage status
OR-Tools	CP-SAT assignment of attractions to trip days.
SQLite and sqlite3	Local session state, chat history and trace storage.
JSON	Approved place profiles, external-response caches and structured state.
Requests	HTTP calls to geocoding, weather and travel-data services.
python-dotenv	Loading configured environment settings.
Scrapy	Optional official-place refresh with a review queue.
OpenStreetMap and Overpass	Geographic objects, attractions, stays, food and source tags.
Nominatim	Place-name geocoding and fallback location resolution.
GeoNames	Supplementary destination-discovery source.
Open-Meteo	Weather forecasts and historical climate estimates.
Ola Maps	Configured geocoding, road matrices, routes and elevation.
Tailwind CSS and PostCSS	Existing frontend styling and CSS processing.
ESLint	Frontend static analysis.
Streamlit Folium and streamlit-folium	Alternative Streamlit interface and map rendering.
google-genai	Declared dependency; shared active client supports other providers.

Road and elevation services require configuration. Cached responses and labelled estimates provide alternatives when live data is unavailable. The React frontend and FastAPI backend are the main application described in this report; Streamlit is an additional entry point in the repository.

CHAPTER 7

DATABASE AND KNOWLEDGE BASE STRUCTURE

CHAPTER 8

MODULES AND IMPLEMENTATION

8.1 Project Folder Structure

[Insert screenshot of the project folder structure here]

Figure 8.1 Project folder structure

The folder structure separates the agent, backend server, web interface, data sources and tests. The agent directory contains NLP and planning modules. The server directory exposes API endpoints, while web contains the React application. The data, scripts and test directories support knowledge maintenance and validation.

8.2 Module Organisation

The following tables list the application's source modules individually, grouped by responsibility. Supporting data, startup scripts and test modules are listed separately. Generated caches, dependency folders and bytecode are runtime artefacts rather than implementation modules.

Windows launch helpers include start.bat, start.ps1, stop.bat, run_backend.bat, run_frontend.bat, free_ports.ps1 and wait_for_backend.ps1. Requirements and package manifests declare Python and web dependencies. Test files document intended checks; their presence alone does not establish that all scenarios pass.

8.3 Conversation NLP and State Modules

Module or File	Implementation responsibility
agent/__init__.py	Initialises the Python agent package.
agent/orchestrator.py	Maintains dialogue history and coordinates routing, model calls and tools.
agent/state.py	Defines structured trip state and traveller preferences.
agent/schemas.py	Declares tool specifications and argument schemas.
agent/prompts.py	Stores conversation instructions and planning prompts.
agent/llm_client.py	Creates provider clients and parses one-shot JSON completions.
agent/nlu.py	Combines entity extraction and intent results.
agent/ner.py	Extracts trip slots using spaCy and regular expressions.
agent/intent.py	Trains and applies TF-IDF plus Logistic Regression intent classification.
agent/intent_rules.py	Recognises high-precision intents through rules.
agent/router.py	Selects NLP or LLM routing using embeddings and state signals.
agent/tool_router.py	Checks prerequisites and dispatches deterministic tool calls.
agent/tools.py	Implements travel tools and updates trip state.
agent/db.py	Saves and reloads SQLite session records.
agent/cache.py	Provides disk-cache support for retrieved planning data.

8.4 Discovery Planning and Route Modules

Module or File	Implementation responsibility
agent/discovery.py	Combines destination sources, filtering and discovery scoring.
agent/nearby_search.py	Filters curated destinations by category and radius.
agent/destinations_db.py	Stores curated destination names, categories and coordinates.
agent/hill_stations.py	Contains hill-station records and specialised nearby search helpers.
agent/geo.py	Resolves place names through Ola and Nominatim geocoding.
agent/pois.py	Retrieves and parses attractions, stays and food from Overpass.
agent/place_profiles.py	Merges approved facts, OSM tags and labelled category defaults.
agent/activities.py	Classifies activity duration and effort with validation and caching.
agent/scheduler.py	Assigns attractions, orders days and builds validated timelines.
agent/route_astar.py	Searches small hotel-to-stops-to-hotel visit sequences.
agent/ola_routing.py	Retrieves road distances and route information.
agent/ola_elevation.py	Retrieves elevation signals for activity assessment.
agent/weather.py	Obtains forecast or historical climate information.
agent/trip_conditions.py	Estimates weather, crowd, risk, season and interest suitability.
agent/opening_hours.py	Checks supported weekly opening-hours expressions.
agent/budget.py	Calculates cost line items from explicit assumptions.
agent/transport.py	Estimates intercity transport choices, time and costs.
agent/stay.py	Handles accommodation-related planning support.
agent/map_view.py	Builds geographic visualisation for the alternative map interface.

8.5 Backend Frontend and Entry Points

Module or File	Implementation responsibility
server/main.py	Defines health, sessions, chat, ideas, itinerary move and map API endpoints.
main.py	Command-line entry point for the agent.
app_streamlit.py	Alternative interactive Streamlit application.
web/src/main.jsx	Mounts the React application in the browser.
web/src/App.jsx	Defines application routing.
web/src/api.js	Wraps communication with the backend API.
web/src/format.js	Provides display-formatting helpers.
web/src/dayColors.js	Defines consistent colours for itinerary days.
pages/Landing.jsx	Displays the landing page and trip-session entry interface.
pages/Builder.jsx	Coordinates the trip-builder experience.
components/ChatPane.jsx	Shows conversation and accepts messages.
components/TripPane.jsx	Organises the trip information panel and tabs.
components/IdeasTab.jsx	Displays attraction and recommendation ideas.
components/ItineraryTab.jsx	Shows daily plans and supports moving itinerary stops.
components/MapTab.jsx	Displays selected places and routes on a map.
components/StayTab.jsx	Presents accommodation-related information.
web/src/index.css	Global styles and frontend styling declarations.
web/src/App.css	Application-level styling.

Page and component paths in this table are relative to web/src. Each interface view consumes backend state rather than duplicating the Python scheduler in the browser.

8.6 Knowledge Maintenance and Test Modules

Module or File	Implementation responsibility
scripts/refresh_official_places.py	Retrieves official-source candidates for human review.
data/india_place_profiles.json	Approved attraction facts and provenance.
data/official_place_sources.json	Source URLs configured for official-place refresh.
official_place_review.json	Generated review queue; used when the refresh script runs.
sessions.db	SQLite session store; location depends on the server working directory.
test/test_api.py	API-related test coverage.
test/test_features.py	Planning feature scenarios.
test/test_nlu.py	Entity and intent understanding scenarios.
test/test_nlp_tool_router.py	Deterministic tool-dispatch scenarios.
test/test_in_between_router.py	Hybrid routing scenarios.
test/test_pipeline.py	End-to-end planning pipeline scenarios.
test/test_schedule_quality.py	Itinerary feasibility and schedule-quality checks.
test/test_multi_select.py	Multiple destination-selection scenarios.
test/test_multi_and_vague.py	Multiple and vague destination requests.
test/test_pagination.py	Paginated recommendation scenarios.
test/test_trip_init_typo.py	Trip initialisation with typo-related inputs.
test/test_geonames.py	GeoNames integration checks.
test/test_llm_budget.py	Language-model and budget interaction checks.
test/test_nvidia.py	NVIDIA provider integration checks.
probe_overpass_status.py	Standalone Overpass-service diagnostic script.

CHAPTER 9

OUTPUT SCREENSHOTS

9.1 Landing page of the AI Travel Agent

[Insert screenshot of landing page and new trip button here]

Figure 9.1 Landing page of the AI Travel Agent

The landing page introduces the travel planner and provides the starting point for a new trip. The traveller opens the planning interface through the new trip button. This screen connects the project overview with the conversational workflow and gives users a clear way to begin entering their travel requirements.

9.2 Saved trip sessions

[Insert screenshot of saved trip cards on the landing page here]

Figure 9.2 Saved trip sessions

Saved trip cards show the destination, origin, dates and current planning stage for earlier sessions. Selecting a card reopens its conversation and trip details. This output illustrates how SQLite persistence supports continued planning, allowing the traveller to return to a trip without entering the same details again.

9.3 Conversational collection of trip preferences

[Insert screenshot of chat showing entered trip details here]

Figure 9.3 Conversational collection of trip preferences

The chat view shows a traveller entering dates, destination, budget and group size in natural language. The agent extracts these details into structured trip state and asks for missing information. This output illustrates how entity extraction supports a conversation that gathers the requirements needed for a travel plan.

9.4 Nearby destination recommendations

[Insert screenshot of chat recommending nearby destinations here]

Figure 9.4 Nearby destination recommendations

The conversation presents destination candidates for a category request such as nearby hill stations. Geographic distance and curated categories help identify relevant choices. The traveller can select a suggestion to continue planning, illustrating how the system handles a trip request before a destination is fixed.

9.5 Attraction recommendations

[Insert screenshot of ideas tab with attraction recommendations here]

Figure 9.5 Attraction recommendations

The Ideas tab lists attractions retrieved for the selected destination and provides information for comparing places. The traveller can review the available options before the itinerary is built. This output connects geographic data and place profiles with the set of candidate activities considered by the scheduler.

9.6 Weather information for the trip dates

[Insert screenshot of chat response showing weather information here]

Figure 9.6 Weather information for the trip dates

The weather response presents conditions for the selected destination and travel dates. It identifies whether the information is a forecast or a historical climate estimate. This output helps the traveller review outdoor activity suitability and understand the weather assumptions that influence the suggested itinerary.

9.7 Transport options and travel estimates

[Insert screenshot of chat response comparing transport options here]

Figure 9.7 Transport options and travel estimates

The transport response compares supported travel modes between the origin and selected destination. Estimated journey time and cost help the traveller assess practical options. This output shows how transport planning uses distance and mode assumptions to support a decision without claiming confirmed fares or bookings.

9.8 Personalised day-wise travel itinerary

[Insert screenshot of generated day-wise itinerary with timings here]

Figure 9.8 Personalised day-wise travel itinerary

The itinerary view groups selected attractions into trip days and displays estimated arrival and departure times. Visit duration, travel time and daily effort limits shape the schedule. This output shows how the planner converts recommended places into an organised sequence of activities that the traveller can review.

9.9 Map visualisation of travel locations

[Insert screenshot of interactive map displaying itinerary places and route here]

Figure 9.9 Map visualisation of travel locations

The map displays itinerary stops at their geographic coordinates and uses day colours to distinguish groups of visits. Route information helps the traveller compare locations with the daily plan. This output makes the distance and sequence relationships visible, supporting a review of how the selected places fit together.

9.10 Accommodation recommendations

[Insert screenshot of stay tab showing accommodation recommendations here]

Figure 9.10 Accommodation recommendations

The Stay tab presents accommodation information for the selected destination. The traveller can compare the details alongside planned attractions and dates. This output shows how stay recommendations form part of the trip workflow, helping the traveller review available information before arranging a confirmed booking.

9.11 Estimated trip cost breakdown

[Insert screenshot of costs tab with total estimate and cost breakdown here]

Figure 9.11 Estimated trip cost breakdown

The Costs tab displays the total trip estimate and its accommodation, food, local travel and activity components. The values use stated assumptions when verified prices are unavailable. This output helps the traveller compare spending categories and assess the proposed plan against the available budget before making bookings.

9.12 Manual itinerary editing

[Insert screenshot of itinerary after moving a stop between days here]

Figure 9.12 Manual itinerary editing

The itinerary view shows a stop after the traveller moves it to another day or changes its position. The backend checks the requested edit and recalculates the affected timeline. This output illustrates how the generated plan can be adjusted through the interface while retaining the scheduling checks used by the application.

9.13 Planning constraints and source information

[Insert screenshot of itinerary details showing intensity and route source here]

Figure 9.13 Planning constraints and source information

The itinerary shows daily intensity, effort and route sources alongside scheduled activities. These fields explain the limits and data assumptions used by the planner. This output helps the traveller distinguish road data from fallback estimates and review the practical reasons behind the arrangement of each trip day.

9.14 Unscheduled attractions and reasons

[Insert screenshot of unscheduled attractions and omission reasons here]

Figure 9.14 Unscheduled attractions and reasons

The unscheduled section lists attractions omitted from the itinerary and explains each omission. Reasons include distance restrictions or limits on the available activity pool. This output helps the traveller understand why a suggested place is absent and review which recommendations could not fit into the generated plan.

9.15 Planning execution trace

[Insert screenshot of trace tab showing planning tool activity here]

Figure 9.15 Planning execution trace

The Trace tab presents tool activity recorded during the trip conversation. It explains which planning operations were invoked as the user supplied details and requested outputs. This screen provides a view of the agent workflow, helping the reviewer follow how the application produced its recommendations and itinerary.

9.16 Multiple destination selection

[Insert screenshot of chat selecting more than one destination here]

Figure 9.16 Multiple destination selection

The conversation shows the traveller choosing more than one destination from the suggested candidates. The agent records the selection and continues the trip dialogue using the updated state. This output illustrates how a single planning session can collect several destinations while preserving the original travel requirements.

9.17 Food and restaurant recommendations

[Insert screenshot of food ideas panel listing restaurants and cuisines here]

Figure 9.17 Food and restaurant recommendations

The food ideas panel lists dining places retrieved near the chosen destination and shows cuisine details when available. These suggestions help the traveller review meal options alongside sightseeing plans. This output illustrates how restaurant information complements attraction recommendations within the same trip workflow.

9.18 Attraction inclusion and exclusion

[Insert screenshot of ideas tab showing included and excluded attractions here]

Figure 9.18 Attraction inclusion and exclusion

The Ideas tab distinguishes included attractions from places excluded by the traveller. Selecting an entry changes its status and requests an updated itinerary. This output shows how direct user preferences affect the planning pool, allowing the traveller to remove unwanted stops without starting a new trip conversation.

9.19 Changing the itinerary pace

[Insert screenshot of chat requesting a relaxed pace and the updated itinerary here]

Figure 9.19 Changing the itinerary pace

The conversation shows a request to make the trip more relaxed and the resulting itinerary update. The planner applies the selected pace to its daily activity limits. This output illustrates how a short natural-language instruction can revise the workload of a trip while keeping the existing destination and travel dates.

9.20 Meal placement in the daily timeline

[Insert screenshot of daily itinerary showing meal stops and timings here]

Figure 9.20 Meal placement in the daily timeline

The daily timeline includes meal stops alongside attraction visits and travel intervals. Their placement helps the traveller review how sightseeing and meals fit within the available day. This output illustrates the scheduler's timeline construction, including the time reserved for activities beyond the main attraction visits.

9.21 Attraction details on the map

[Insert screenshot of map with an attraction marker popup open here]

Figure 9.21 Attraction details on the map

An open map marker popup identifies a selected attraction within its geographic setting. The surrounding markers help the traveller relate this place to other stops on the route. This output illustrates how the map supports closer inspection of individual locations alongside the broader view of the planned daily journey.

9.22 Place facts and source provenance

[Insert screenshot of attraction card showing entry fees opening hours and source here]

Figure 9.22 Place facts and source provenance

An attraction card shows available entry fees, opening hours and source information next to the planned visit. These details help the traveller review practical conditions before travelling. This output illustrates how the interface presents sourced facts alongside estimates, with missing information requiring further verification.

9.23 Extracted trip slots and intent

[Insert screenshot of trace entry showing extracted entities and intent here]

Figure 9.23 Extracted trip slots and intent

The NLU trace entry displays the recognised intent and trip details extracted from a user message. It connects the wording in the conversation with structured values used by the planner. This output helps the reviewer inspect entity extraction and intent classification without treating a displayed prediction as measured accuracy.

9.24 NLP and LLM routing decision

[Insert screenshot of trace entry showing the nlp or llm routing decision here]

Figure 9.24 NLP and LLM routing decision

A routing trace entry records whether the message was directed to the NLP tool engine or the LLM path. The available decision details help explain how the hybrid router selected its next action. This output illustrates the link between language understanding, dialogue state and the planning operation used to answer the request.

9.25 Validation of an itinerary edit

[Insert screenshot of itinerary showing a validation message for a rejected move here]

Figure 9.25 Validation of an itinerary edit

The itinerary displays a validation message when a requested stop movement cannot be accepted. The traveller can review the response and choose a different adjustment. This output illustrates how editing checks protect the plan from unsuitable changes, rather than accepting every movement without checking the affected schedule.

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENT

10.1 Conclusion

The AI Travel Agent brings conversational input, geographic data and itinerary optimisation into one application. NLP modules extract travel requirements and recognise intents, while the hybrid router selects direct tool execution or LLM reasoning according to the message and dialogue state. The planning tools retrieve destinations and attractions, estimate activity properties and construct day-wise schedules through CP-SAT.

Route ordering and map views make the resulting plan easier to inspect. SQLite persistence lets the user continue an existing trip, and provenance labels help distinguish approved facts, external tags and estimates. The project demonstrates how language understanding can connect to deterministic algorithms that enforce explicit constraints rather than relying on a generated paragraph alone.

10.2 Current Limitations

The planner depends on the completeness of source data and the availability of external services. Curated attraction coverage is limited, complex opening-hour expressions are not fully verified, and activity, crowd and budget values may be estimates. Long-range weather uses historical climate data. Route quality depends on the available distance matrix, and the scheduler's feasibility model does not guarantee booking availability or total budget compliance.

10.3 Future Enhancement

Future work can add multilingual entity extraction and a larger labelled intent dataset evaluated on a separate test set. More manually verified attraction profiles would improve duration, fees, accessibility and opening-hours coverage. Live hotel and ticket integrations could replace assumption-based prices with available quotations and support explicit budget constraints.

Further improvements include traffic-aware travel times, feedback-based recommendations, collaborative group planning and mobile access. Evaluation should measure entity extraction, intent classification, route cost, constraint violations and completed user journeys. These measurements would provide evidence for improvements beyond the current implementation description.

REFERENCES AND SOURCE MATERIALS

The implementation descriptions and numerical assumptions in this report are based on the project source files listed below. Package names identify the software used; no performance measurements are inferred from the existence of a dependency or test file.

- [1] Project README.md and requirements.txt: application overview, dependency declarations and data-source conventions.
- [2] web/package.json and web/src: React interface, mapping packages and frontend component structure.
- [3] server/main.py: session, chat, recommendation, map and itinerary-edit API endpoints.
- [4] agent/ner.py, agent/nlu.py, agent/intent.py and agent/intent_rules.py: entity rules, intent training examples, TF-IDF and confidence handling.
- [5] agent/router.py and agent/tool_router.py: semantic routing, TF-IDF fallback and deterministic dispatch.
- [6] agent/orchestrator.py, agent/llm_client.py, agent/schemas.py and agent/prompts.py: dialogue flow and language-model integration.
- [7] agent/scheduler.py and agent/route_astar.py: attraction assignment, objective terms, route sequencing and timeline checks.
- [8] agent/budget.py and agent/transport.py: documented cost assumptions and transport estimates.
- [9] agent/pois.py, agent/place_profiles.py, agent/weather.py and agent/trip_conditions.py: external records, provenance and suitability estimates.
- [10] agent/db.py, agent/state.py and scripts/refresh_official_places.py: persistence, state models and reviewed knowledge maintenance.